How to Reproduce the PHAGENT_AMR Pipeline

This document provides step-by-step instructions to reproduce the assembly-based AMR profiling pipeline from scratch.

Prerequisites

Required Tools

Tool	Version	Purpose
Python	3.9+	Script execution
Flye	>= 2.9	Metagenome assembly
Medaka	latest	Consensus polishing
AMRFinderPlus	latest	AMR gene detection
MOB-suite	3.1.9	Plasmid classification
pandas	latest	Data processing

Install via Conda/Mamba

```
# Create environment
conda create -n phagent_amr -c bioconda -c conda-forge \
    python=3.11 flye medaka minimap2 samtools bcftools \
    ncbi-amrfinderplus mob_suite=3.1.9 pandas openpyxl -y

# Activate
conda activate phagent_amr
```

Verify Installation

```
flye --version
medaka --version
amrfinder --version
mob_recon --help | head -3
python -c "import pandas; print('pandas', pandas.__version__)"
```

Required Files

Ensure you have the following source files:

File	Location	Description
Raw FASTQ data	`PHAGENT_AMR/Input/WW_FarmUAA_RLB_20250219-fastqs.zip`	Nanopore sequencing reads
Farm metadata	`output/basic/barcode_farm_mapping.csv`	Barcode-to-farm mapping (from main pipeline)

Step 1: Prepare Input Data

1.1 Extract FASTQ Files

```
cd PHAGENT_AMR
mkdir -p data/fastq
cd data/fastq
unzip ../../Input/WW_FarmUAA_RLB_20250219-fastqs.zip
```

1.2 Verify Input Data

```
ls data/fastq/barcode*.fastq.gz | wc -l
# Expected: up to 24 files (barcode01-barcode24)

# Check file sizes
ls -lh data/fastq/barcode*.fastq.gz
```

1.3 Verify Farm Metadata

```
head -5 ../output/basic/barcode_farm_mapping.csv
# Expected columns:
barcode,sample_id,farm_type,farm_id,sample_type,oblast,collection_period

wc -l ../output/basic/barcode_farm_mapping.csv
# Expected: 25 (24 records + header)
```

If `barcode_farm_mapping.csv` does not exist, run the main pipeline first:

```
cd ..  # back to bio-phagent root
python scripts/create_output_mappings.py
```

Step 2: Metagenome Assembly (Flye)

2.1 Run Assembly

```
cd PHAGENT_AMR

# For each barcode with FASTQ data:
for fastq in data/fastq/barcode*.fastq.gz; do
    barcode=$(basename "$fastq" .fastq.gz)
    outdir="results/assembly/${barcode}"

    if [ -f "${outdir}/assembly.fasta" ]; then
        echo "Skipping ${barcode} (already assembled)"
        continue
    fi

    echo "Assembling ${barcode}..."
    mkdir -p "$outdir"
    flye --nano-raw "$fastq" --out-dir "$outdir" --meta --threads 4
done
```

2.2 Verify Assembly

```
# Check which barcodes produced assemblies
for dir in results/assembly/barcode*/; do
    barcode=$(basename "$dir")
    if [ -f "${dir}/assembly.fasta" ]; then
        contigs=$(grep -c "^>" "${dir}/assembly.fasta")
        size=$(wc -c < "${dir}/assembly.fasta")
        echo "${barcode}: ${contigs} contigs, ${size} bytes"
    else
        echo "${barcode}: NO ASSEMBLY"
    fi
done
```

Expected results for validated barcodes:

Barcode	Contigs	Total Size
barcode07	4	~5.6 KB
barcode08	4	~13 KB
barcode09	6	~35 KB
barcode18	26	~106 KB

Note: Some barcodes may produce empty assemblies (0 contigs). This is expected for low-coverage samples. Known empty assemblies: barcode03, barcode11, barcode23, barcode24.

Step 3: Consensus Polishing (Medaka)

3.1 Run Polishing

```
MEDAKA_MODEL="r1041_e82_400bps_sup_v5.2.0"

for assembly in results/assembly/barcode*/assembly.fasta; do
    barcode=$(basename "$(dirname "$assembly")")
    fastq="data/fastq/${barcode}.fastq.gz"
    outdir="results/polished/${barcode}"

    if [ -f "${outdir}/consensus.fasta" ]; then
        echo "Skipping ${barcode} (already polished)"
        continue
    fi

    if [ ! -f "$fastq" ]; then
        echo "Skipping ${barcode} (no FASTQ file)"
        continue
    fi

    # Skip empty assemblies
    if [ ! -s "$assembly" ]; then
        echo "Skipping ${barcode} (empty assembly)"
        continue
    fi

    echo "Polishing ${barcode}..."
    mkdir -p "$outdir"
    medaka_consensus -i "$fastq" -d "$assembly" -o "$outdir" \
        -m "$MEDAKA_MODEL" -t 4
done
```

3.2 Verify Polishing

```
for dir in results/polished/barcode*/; do
    barcode=$(basename "$dir")
    if [ -f "${dir}/consensus.fasta" ]; then
        contigs=$(grep -c "^>" "${dir}/consensus.fasta")
        echo "${barcode}: ${contigs} polished contigs"
    fi
done
```

Expected results for validated barcodes:

Barcode	Polished Contigs	Size
barcode07	2	~5.5 KB
barcode08	4	~13 KB
barcode09	10	~35 KB
barcode18	29	~104 KB

Step 4: AMR Gene Detection (AMRFinderPlus)

4.1 Update AMRFinderPlus Database

```
amrfinder -u
```

4.2 Run AMRFinderPlus

```
mkdir -p results/amr TSV

for consensus in results/polished/barcode*/consensus.fasta; do
    barcode=$(basename "$(dirname "$consensus")")
    output="TSV/${barcode}_amr.tsv"

    if [ -f "$output" ]; then
        echo "Skipping ${barcode} (already analyzed)"
        continue
    fi

    echo "Running AMRFinderPlus on ${barcode}..."
    amrfinder -n "$consensus" -o "$output"
done
```

4.3 Verify AMR Results

```
for tsv in TSV/barcode*_amr.tsv; do
    barcode=$(basename "$tsv" _amr.tsv)
    genes=$(($(wc -l < "$tsv") - 1))
```

```
    echo "${barcode}: ${genes} AMR gene(s)"
done
```

Expected results for validated barcodes:

Barcode	AMR Genes	Key Finding
barcode07	1	qnrD1 (QUINOLONE, 100% identity)
barcode08	0	No AMR genes detected
barcode09	1	qnrD (QUINOLONE, 83.64% coverage)
barcode18	1	qnrD1 (QUINOLONE, 100% identity)

Step 5: Plasmid Classification (MOB-suite)

5.1 Run MOB-recon

```
for assembly in results/assembly/barcode*/assembly.fasta; do
    barcode=$(basename "$(dirname "$assembly")")
    outdir="mob_results/${barcode}"

    if [ -f "${outdir}/contig_report.txt" ]; then
        echo "Skipping ${barcode} (already classified)"
        continue
    fi

    if [ ! -s "$assembly" ]; then
        echo "Skipping ${barcode} (empty assembly)"
        continue
    fi

    echo "Running MOB-suite on ${barcode}..."
    mkdir -p "$outdir"
    mob_recon -i "$assembly" -o "$outdir"
done
```

5.2 Verify MOB-suite Results

```
for dir in mob_results/barcode*/; do
    barcode=$(basename "$dir")
    if [ -f "${dir}/contig_report.txt" ]; then
        total=$(($(wc -l < "${dir}/contig_report.txt") - 1))
        plasmids=$(grep -c "plasmid" "${dir}/contig_report.txt" || true)
        echo "${barcode}: ${total} contigs classified, ${plasmids} on
plasmids"
```

```
    fi
done
```

Expected results for validated barcodes:

Barcode	Total Contigs	Plasmid Contigs	Nearest Neighbor
barcode07	2	1 (contig_2)	Proteus mirabilis
barcode08	4	0	-
barcode09	10	2 (contig_4, contig_7)	Proteus mirabilis, Salmonella enterica
barcode18	29	1 (contig_20)	Proteus mirabilis

Step 6: Merge Results

6.1 Run Merge Script

```
python scripts/05_merge_results.py
```

6.2 Verify Merge Output

```
# Check combined table exists
ls -la results/merged/all_samples_amr_annotated.tsv

# Check row count
wc -l results/merged/all_samples_amr_annotated.tsv
# Expected: 4 (3 AMR genes + header, for 4-barcode dataset)

# Check per-barcode files
ls results/merged/barcode*_amr_annotated.tsv

# Verify key columns present
head -1 results/merged/all_samples_amr_annotated.tsv | tr '\t' '\n' | grep -E
"is_priority|is_esbl|is_plasmid|farm_type|molecule_type"
# Expected: is_priority, is_esbl, is_plasmid_borne, farm_type, molecule_type
```

6.3 Validation Checkpoint

Verify that barcode07 qnrD1 on contig_2 is correctly annotated:

```
python -c "
import pandas as pd
df = pd.read_csv('PHAGENT_AMR/results/merged/all_samples_amr_annotated.tsv',
sep='\t')
row = df[df['barcode'] == 'barcode07'].iloc[0]
assert row['Element symbol'] == 'qnrD1', f'Expected qnrD1, got {row[\"Element
symbol\"]}'
assert row['Contig id'] == 'contig_2', f'Expected contig_2, got {row[\"Contig
id\"]}'
assert row['is_priority'] == True, 'Expected is_priority=True'
assert row['is_plasmid_borne'] == True, 'Expected is_plasmid_borne=True'
assert row['farm_type'] == 'Poultry', f'Expected Poultry, got
{row[\"farm_type\"]}'
assert row['farm_id'] == 'poultry #3', f'Expected poultry #3, got
{row[\"farm_id\"]}'
print('All merge validations passed.')
"
```

Expected Console Output

```
Loading AMR results...
  3 AMR gene detections from 3 barcodes
Loading MOB-suite results...
  46 contig classifications from 4 barcodes
Loading farm metadata...
  24 barcode-to-farm mappings
Merging datasets...
  Combined table: .../results/merged/all_samples_amr_annotated.tsv (3 rows)
  barcode07: 1 genes -> barcode07_amr_annotated.tsv
  barcode09: 1 genes -> barcode09_amr_annotated.tsv
  barcode18: 1 genes -> barcode18_amr_annotated.tsv

Merge complete: 3 total AMR genes, 3 priority, 0 ESBL, 3 plasmid-borne
```

Step 7: Generate Reports

7.1 Run Report Script

```
python scripts/06_report.py
```

7.2 Verify Report Outputs

```
# Check all report files created
ls -la results/reports/

# Verify row counts
echo "Sample summary rows: $(wc -l <
results/reports/amr_summary_by_sample.csv)"
# Expected: 5 (4 barcodes + header)

echo "Priority findings rows: $(wc -l <
results/reports/priority_findings.csv)"
# Expected: 4 (3 priority hits + header)

echo "Cross-sample rows: $(wc -l < results/reports/cross_sample_summary.csv)"
# Expected: 3 (2 unique genes + header)

echo "Farm summary rows: $(wc -l < results/reports/farm_level_summary.csv)"
# Expected: 4 (3 farms + header)
```

7.3 Verify Sample Summary

```
# Check barcode08 has 0 genes (no AMR detected)
python -c "
import pandas as pd
df = pd.read_csv('PHAGENT_AMR/results/reports/amr_summary_by_sample.csv')
bc08 = df[df['barcode'] == 'barcode08'].iloc[0]
assert bc08['total_amr_genes'] == 0, 'barcode08 should have 0 AMR genes'
print('barcode08 correctly reported with 0 AMR genes')
print(df.to_string(index=False))
"
```

7.4 Verify Priority Findings

All 3 detected genes should appear as priority findings (QUINOLONE class):

```
python -c "
import pandas as pd
df = pd.read_csv('PHAGENT_AMR/results/reports/priority_findings.csv')
assert len(df) == 3, f'Expected 3 priority findings, got {len(df)}'
assert all(df['Class'] == 'QUINOLONE'), 'All findings should be QUINOLONE
class'
print(f'{len(df)} priority findings, all QUINOLONE.')
print(df[['barcode', 'Element symbol', 'Contig id',
```

```
'is_plasmid_borne']].to_string(index=False))
"
```

Expected Console Output

```
Loading merged AMR data...
  3 annotated AMR records, 4 analyzed barcodes
Generating per-sample summary...
  -> amr_summary_by_sample.csv (4 rows)
Generating priority findings...
  -> priority_findings.csv (3 rows)
Generating cross-sample summary...
  -> cross_sample_summary.csv (2 rows)
Generating farm-level summary...
  -> farm_level_summary.csv (3 rows)

============================================================
AMR REPORT SUMMARY
============================================================

Samples analyzed:     4
Samples with AMR:     3
Total AMR genes:      3
Priority findings:    3
Plasmid-borne:        3

Priority AMR detections:
  - qnrD1 (QUINOLONE) in barcode07 on contig_2 [PLASMID]
  - qnrD (QUINOLONE) in barcode09 on contig_4 [PLASMID]
  - qnrD1 (QUINOLONE) in barcode18 on contig_20 [PLASMID]

Recurring genes across samples:
  - qnrD1 (QUINOLONE): 2 samples, 2 farms
  - qnrD (QUINOLONE): 1 samples, 1 farms

============================================================
```

Output Schema Reference

all_samples_amr_annotated.tsv

Column	Source	Description
Protein id	AMRFinderPlus	Protein accession (NA for nucleotide hits)
Contig id	AMRFinderPlus	Contig where gene was found
Start, Stop, Strand	AMRFinderPlus	Gene coordinates on contig

Column	Source	Description
Element symbol	AMRFinderPlus	Gene name (e.g. qnrD1)
Element name	AMRFinderPlus	Full gene description
Class, Subclass	AMRFinderPlus	Resistance class (e.g. QUINOLONE)
Method	AMRFinderPlus	Detection method (ALLELEX, PARTIALX, etc.)
% Coverage of reference	AMRFinderPlus	Alignment coverage
% Identity to reference	AMRFinderPlus	Sequence identity
barcode	Derived	Sample identifier (e.g. barcode07)
molecule_type	MOB-suite	chromosome or plasmid
primary_cluster_id	MOB-suite	Plasmid cluster (e.g. AB434)
predicted_mobility	MOB-suite	Conjugative, mobilizable, or non-mobilizable
mash_nearest_neighbor	MOB-suite	Closest known plasmid
mash_neighbor_identification	MOB-suite	Species of nearest neighbor
sample_id	Metadata	Original sample ID
farm_type	Metadata	Pig or Poultry
farm_id	Metadata	Farm identifier
sample_type	Metadata	Native or Enriched
oblast	Metadata	Ukrainian region
is_priority	Derived	True if Class in BETA-LACTAM, CARBAPENEM, COLISTIN, QUINOLONE
is_esbl	Derived	True if gene starts with “bla”
is_plasmid_borne	Derived	True if molecule_type is plasmid

amr_summary_by_sample.csv

Column	Description
barcode	Sample identifier
sample_id, farm_type, farm_id, sample_type, oblast	Farm metadata

Column	Description
total_amr_genes	Number of AMR genes detected
unique_amr_genes	Number of distinct gene symbols
unique_classes	Number of distinct resistance classes
priority_gene_count	Genes in priority classes
esbl_gene_count	ESBL genes (bla-prefix)
plasmid_borne_count	Genes on plasmid contigs
gene_list	Comma-separated list of gene symbols

Troubleshooting

Prerequisites Check

```
# Check conda environment
conda activate phagent_amr
which flye medaka amrfinder mob_recon python
```

Common Issues

Issue	Solution
`FileNotFoundError: barcode_farm_mapping.csv`	Run main pipeline `create_output_mappings.py` first
`FileNotFoundError: all_samples_amr_annotated.tsv`	Run `05_merge_results.py` before `06_report.py`
Flye produces 0 contigs	Expected for low-coverage samples; pipeline handles gracefully
Medaka model not found	Check `medaka --list_models` for available models; update `MEDAKA_MODEL`
AMRFinderPlus database error	Run `amrfinder -u` to update database
MOB-suite database error	Run `mob_init` to initialize databases
Empty TSV (header only)	No AMR genes detected in that sample; this is valid

Re-running Steps

Each step is idempotent. To re-run a specific step, delete its outputs first:

```
# Re-run assembly for barcode07
rm -rf results/assembly/barcode07/
```

```bash
# Then re-run Step 2

# Re-run merge from scratch
rm -rf results/merged/
python scripts/05_merge_results.py

# Re-run reports from scratch
rm -rf results/reports/
python scripts/06_report.py
```

Comparing with Existing Results

If you have previously generated results in `Assembly/`, `Medaka from assembly/`, and `TSV/`:

```bash
# Compare assembly sizes
for bc in barcode07 barcode08 barcode09 barcode18; do
    old_size=$(wc -c < "Assembly/${bc}_assembly.fasta" 2>/dev/null || echo
"missing")
    new_size=$(wc -c < "results/assembly/${bc}/assembly.fasta" 2>/dev/null ||
echo "missing")
    echo "${bc}: old=${old_size} new=${new_size}"
done

# Compare AMR detections
for bc in barcode07 barcode08 barcode09 barcode18; do
    old_genes=$(($(wc -l < "TSV/${bc}_amr.tsv") - 1))
    new_genes=$(($(wc -l < "results/amr/${bc}_amr.tsv" 2>/dev/null || echo
"1") - 1))
    echo "${bc}: old=${old_genes} genes, new=${new_genes} genes"
done
```

Complete Pipeline Script

Save as `run_pipeline.sh` and run with `bash run_pipeline.sh`:

```bash
#!/bin/bash
set -euo pipefail

echo "========================================"
echo "PHAGENT_AMR Pipeline"
echo "========================================"

cd "$(dirname "$0")"
```

```bash
# Step 1: Assembly
echo ""
echo "[Step 1/6] Metagenome assembly (Flye)..."
for fastq in data/fastq/barcode*.fastq.gz; do
    barcode=$(basename "$fastq" .fastq.gz)
    outdir="results/assembly/${barcode}"
    [ -f "${outdir}/assembly.fasta" ] && continue
    mkdir -p "$outdir"
    flye --nano-raw "$fastq" --out-dir "$outdir" --meta --threads 4
done

# Step 2: Polishing
echo ""
echo "[Step 2/6] Consensus polishing (Medaka)..."
MEDAKA_MODEL="r1041_e82_400bps_sup_v5.2.0"
for assembly in results/assembly/barcode*/assembly.fasta; do
    barcode=$(basename "$(dirname "$assembly")")
    fastq="data/fastq/${barcode}.fastq.gz"
    outdir="results/polished/${barcode}"
    [ -f "${outdir}/consensus.fasta" ] && continue
    [ ! -f "$fastq" ] && continue
    [ ! -s "$assembly" ] && continue
    mkdir -p "$outdir"
    medaka_consensus -i "$fastq" -d "$assembly" -o "$outdir" -m
"$MEDAKA_MODEL" -t 4
done

# Step 3: AMR detection
echo ""
echo "[Step 3/6] AMR gene detection (AMRFinderPlus)..."
amrfinder -u
mkdir -p TSV
for consensus in results/polished/barcode*/consensus.fasta; do
    barcode=$(basename "$(dirname "$consensus")")
    output="TSV/${barcode}_amr.tsv"
    [ -f "$output" ] && continue
    amrfinder -n "$consensus" -o "$output"
done

# Step 4: Plasmid classification
echo ""
echo "[Step 4/6] Plasmid classification (MOB-suite)..."
for assembly in results/assembly/barcode*/assembly.fasta; do
    barcode=$(basename "$(dirname "$assembly")")
    outdir="mob_results/${barcode}"
    [ -f "${outdir}/contig_report.txt" ] && continue
    [ ! -s "$assembly" ] && continue
    mkdir -p "$outdir"
```

```
    mob_recon -i "$assembly" -o "$outdir"
done

# Step 5: Merge
echo ""
echo "[Step 5/6] Merging results..."
python scripts/05_merge_results.py

# Step 6: Reports
echo ""
echo "[Step 6/6] Generating reports..."
python scripts/06_report.py

echo ""
echo "========================================"
echo "Pipeline complete!"
echo "========================================"
```

Final Output Directory

```
PHAGENT_AMR/
    results/
        assembly/
            barcode{XX}/assembly.fasta        # Flye metagenome assembly
            barcode{XX}/assembly_info.txt      # Assembly statistics
        polished/
            barcode{XX}/consensus.fasta        # Medaka polished consensus
        merged/
            all_samples_amr_annotated.tsv      # Combined annotated AMR table
            barcode{XX}_amr_annotated.tsv      # Per-barcode annotated tables
        reports/
            amr_summary_by_sample.csv          # Per-sample gene counts +
metadata
            priority_findings.csv              # ESBL / colistin / carbapenem
hits
            cross_sample_summary.csv           # Gene-centric cross-sample
view
            farm_level_summary.csv             # Farm-centric aggregation
    TSV/
        barcode{XX}_amr.tsv                    # Raw AMRFinderPlus output per
barcode
    mob_results/
        barcode{XX}/contig_report.txt          # MOB-suite contig
classification
        barcode{XX}/chromosome.fasta           # Predicted chromosome
sequences
        barcode{XX}/plasmid_*.fasta            # Predicted plasmid sequences
```